

	GUÍA DE TRABAJO PRÁCTICO - EXPERIMENTAL Talleres y Laboratorios de Docencia ITM	Código	FGL 029
		Versión	03
		Fecha	18-07-2023

1. IDENTIFICACIÓN DE LA GUÍA

Nombre de la guía:	Taller 2
Código de la guía (No.):	002
Taller(es) o Laboratorio(s) aplicable(s):	
Tiempo de trabajo práctico estimado:	8 horas
Asignatura(s) aplicable(s):	Programación Avanzada
Programa(s) Académico(s) / Facultad(es):	Ingeniería Electrónica / Ingeniería de Telecomunicaciones

COMPETENCIAS	CONTENIDO TEMÁTICO	INDICADOR DE LOGRO
C2: Capacidad para aplicar el diseño de ingeniería para generar soluciones que satisfagan necesidades específicas, considerando factores globales y económicos. C5: Capacidad para reconocer responsabilidades profesionales en situaciones de ingeniería y emitir juicios fundamentados al analizar e interpretar datos	Implementación de funciones con parámetros fijos, valores por defecto, y argumentos variables (*args y **kwargs). Creación de clases y objetos, atributos y métodos, herencia mediante super(), sobrescritura de métodos, polimorfismo, encapsulación, miembros privados, métodos getter y setter y sobrecarga de operadores.	Diseña y analiza clases aplicando encapsulación, herencia, polimorfismo y sobrecarga de operadores para representar y resolver problemas de ingeniería. Aplica la lógica de programación para organizar grandes volúmenes de datos y modularizar soluciones algorítmicas complejas.

2. FUNDAMENTO TEÓRICO

El desarrollo de soluciones de software en ingeniería requiere no solo conocer las estructuras básicas de programación, sino también organizar el código de manera clara, reutilizable y fácil de mantener. En este contexto, las funciones permiten dividir un problema en tareas más pequeñas y específicas, facilitando la reutilización de código mediante el uso de parámetros, valores por defecto y argumentos variables.

La Programación Orientada a Objetos (POO) permite representar entidades mediante clases que agrupan atributos y métodos. A partir de este paradigma es posible construir soluciones más estructuradas mediante conceptos como la herencia, que permite reutilizar y extender características de otras clases; el polimorfismo, que posibilita que diferentes objetos respondan de manera distinta ante una misma operación; y la encapsulación, que permite controlar el acceso y modificación de los atributos internos de un objeto mediante miembros privados y métodos getter y setter.

Python permite además personalizar el comportamiento de los objetos mediante métodos especiales, haciendo posible implementar operaciones como la representación de objetos y la sobrecarga de operadores. Estas herramientas permiten desarrollar modelos computacionales más cercanos a los sistemas y componentes encontrados en aplicaciones de ingeniería.

Finalmente, la capacidad de leer, interpretar y depurar código constituye una competencia fundamental en programación. Analizar la ejecución de un programa, identificar errores y proponer correcciones permite fortalecer la comprensión de los conceptos y desarrollar soluciones más confiables y eficientes.

3. OBJETIVO(S)

	GUÍA DE TRABAJO PRÁCTICO - EXPERIMENTAL Talleres y Laboratorios de Docencia ITM	Código	FGL 029
		Versión	03
		Fecha	18-07-2023

3.1. Objetivo general

Desarrollar soluciones de programación en Python mediante el uso de funciones y principios de Programación Orientada a Objetos, integrando clases, métodos, herencia, polimorfismo, encapsulación y sobrecarga de operadores, así como estrategias de lectura, análisis y depuración de código.

3.2. Objetivos específicos

- Construir funciones utilizando parámetros fijos, valores por defecto, *args y **kwargs, aplicando validaciones y retornando resultados que puedan ser reutilizados en otros procesos.
- Diseñar clases que integren atributos, métodos de instancia y métodos especiales para representar entidades y resolver problemas básicos de ingeniería.
- Aplicar herencia y el uso de super() para reutilizar y extender el comportamiento de clases existentes.
- Implementar polimorfismo mediante la sobrescritura de métodos, identificando cómo una misma operación puede producir comportamientos diferentes según el tipo de objeto.
- Aplicar principios de encapsulación mediante atributos privados y métodos getter y setter para controlar el acceso y modificación de la información de un objeto.
- Implementar sobrecarga de operadores mediante métodos especiales de Python para definir operaciones entre objetos creados por el usuario.
- Analizar la ejecución de programas orientados a objetos e identificar errores de implementación, lógica o diseño mediante ejercicios de lectura y depuración de código.

4. RECURSOS REQUERIDOS

Los elementos requeridos para realizar el taller son:

- Computador personal con internet.
- Acceso a software tipo Visual Studio Code, Google Colab, Kaggle Notebooks, entre otros.
- Material de estudio entregado en clase (Notebooks).

5. ASPECTOS DE SEGURIDAD

No aplica.

6. PROCEDIMIENTO O METODOLOGÍA PARA EL DESARROLLO

6.1. Como líder del semillero de investigación, necesitas una función que valide si tres mediciones de resistencia de un material son consistentes antes de guardarlas en la base de datos:

- Define una variable fuera de la función llamada LIMITE_VARIACION = 20.5. Esta representará la diferencia máxima permitida entre la lectura más alta y la más baja.
- Crea la función validar_ensayo(id_prueba, m1, m2, m3, **metadatos), donde:
 - id_prueba: Un identificador (string).
 - m1, m2, m3: Tres parámetros fijos (float) que representan las mediciones.
 - **metadatos: Argumentos variables con nombre (ej. operador, laboratorio, fecha).
- Verifica que m1, m2, m3 sean números. Si no, debe retornar un mensaje de error.
- Calcula el promedio y la diferencia entre el valor máximo y el mínimo de las tres lecturas.
- La función debe comparar esa diferencia con la variable global LIMITE_VARIACION.

	GUÍA DE TRABAJO PRÁCTICO - EXPERIMENTAL Talleres y Laboratorios de Docencia ITM	Código	FGL 029
		Versión	03
		Fecha	18-07-2023

- f) La función debe entregar un diccionario que contenga:
- El `id_prueba` y el promedio calculado.
 - Un valor booleano aprobado (True si la diferencia es menor o igual al límite global, False de lo contrario).
 - Todos los datos contenidos en `**metadatos` integrados en el mismo diccionario

6.2. Crea una clase llamada `Curso` que permita gestionar la información básica de las asignaturas que dictas.

- El constructor debe recibir los atributos de instancia `nombre` (str), el `codigo` (str) y el número de créditos (int).
- Personaliza la salida para que, al imprimir el objeto, se vea así: "Asignatura: [Nombre] | Código: [Código] | Créditos: [N]".
- Crea un método llamado `horas_estudio()` que retorne el número de horas de estudio independiente recomendadas (calculadas como créditos $\times$ 3).
- Crea dos objetos (instancia) correspondientes a dos cursos ("Programación Avanzada" y "Lógica de programación") e imprime sus datos y sus respectivas horas de estudio.

6.3. En un sistema de adquisición de datos se reciben varias mediciones de voltaje. Diseña una función llamada `analizar_mediciones(*mediciones, limite=5.0)`. La función debe:

- Recibir cualquier cantidad de mediciones mediante `*mediciones`.
- Calcular el promedio de los valores recibidos.
- Contar cuántas mediciones son mayores que `limite`.
- Retornar dos valores: el promedio y la cantidad de mediciones que superan el límite.
- Si no se reciben mediciones, debe retornar 0.0 y 0.
- Pruebe la función usando al menos dos cantidades diferentes de argumentos y modifique el valor de `limite` en una de las llamadas.

6.4. Diseña una clase llamada `Sensor` que represente un sensor utilizado en un sistema de monitoreo. La clase debe contener:

- Los atributos de instancia: `id_sensor`, `tipo`, `unidad`, `valor`
- Un método `actualizar(valor)` que permita modificar la lectura almacenada.
- Un método `esta_en_rango(minimo, maximo)` que retorne True cuando la lectura se encuentre dentro del intervalo indicado y False en caso contrario.
- Personalice la representación del objeto mediante `__str__()` para mostrar una salida similar a:

Sensor S01 | Temperatura: 28.5 °C
- Cree al menos dos objetos y pruebe sus métodos.

6.5. Especialización de Usuarios (Herencia y `super()`)

- Define una clase `Persona` (padre) que reciba `nombre` e `id_escolar` en su constructor.
- Define la clase (hija) `Estudiante`, que debe heredar de `Persona`. Utiliza `super()` para pasar el nombre y el ID de la clase padre. Además, inicializa de forma interna una lista vacía llamada `self.notas`.

- c) Agrega el método `agregar_nota(nota)`. Este método debe recibir un valor numérico y utilizar el método `append()` para guardarlo en la lista `self.notas`.
 - d) La clase debe retornar una cadena con el nombre del estudiante y su promedio académico (usar `__str__`). Si el estudiante no tiene notas, el método debe indicar que el promedio es 0.0 para evitar errores de división por cero.
- 6.6.** Crear una clase padre llamada `Instrumento` con atributos `marca`, `modelo` y un método `informacion()`. Posteriormente cree las clases: `Multimetro` y `Osciloscopio`. Ambas deben heredar de `Instrumento`.
- a) Utilice `super()` para inicializar los atributos heredados.
 - b) `Multimetro` debe añadir el atributo `resolucion`.
 - c) `Osciloscopio` debe añadir el atributo `ancho_banda`.
 - d) Cree al menos una instancia de cada clase.
- 6.7.** A partir del ejercicio anterior, agregue a la clase `Instrumento` el método `medir()`, que retorne "Medición genérica".
- a) Sobrescriba este método en las subclases, de forma que `Multimetro` retorne "Midiendo voltaje" y `Osciloscopio` retorne "Capturando señal".
 - b) Almacene diferentes objetos en una lista.
 - c) Recorra la lista utilizando un ciclo.
 - d) Ejecute el método `medir()` para cada objeto.
 - e) Explique brevemente por qué la instrucción `instrumento.medir()` puede producir resultados diferentes.
- 6.8.** Diseñe una clase llamada `FuenteDC` que represente una fuente de alimentación de laboratorio, que debe contener: `Marca`, `voltaje` y `__corriente_maxima`, donde `__corriente_maxima` debe ser un atributo privado.
- a) Cree un método getter `get_corriente_maxima()` que permita consultar el valor.
 - b) Cree un método setter `set_corriente_maxima(valor)` que permita modificarlo únicamente cuando `valor > 0`.
 - c) Si se intenta asignar un valor menor o igual que cero, conserve el valor anterior y muestre un mensaje de error.
 - d) Cree una instancia y pruebe tanto el getter como el setter.
 - e) Intente acceder directamente al atributo mediante `fuente.__corriente_maxima` y explique lo que sucede. Compare este resultado con el acceso realizado mediante el getter.
- 6.9.** Suponga que, para tus proyectos de investigación, necesitas gestionar presupuestos y para ello va a implementar sobrecarga de operadores. Crea una clase llamada `Presupuesto`.
- a) Recibe un valor numérico llamado `monto`.
 - b) Permite sumar dos objetos de tipo `Presupuesto`. La operación debe retornar una nueva instancia de `Presupuesto` con la suma de ambos montos.
 - c) Implementa el método necesario para poder usar el operador `>` (mayor que) entre dos presupuestos, comparando sus montos.

- d) Prueba: Crea el `presupuesto_investigacion` (5.000.000) y el `presupuesto_equipos` (3.500.000). Súmalos para obtener un `presupuesto_total` y verifica mediante una sentencia `if` cuál de los dos presupuestos originales es mayor.

6.10. Crear una clase `Medicion` que contenga valor y unidad. Sobrecargue el operador `+` utilizando `__add__()`. Dos mediciones pueden sumarse únicamente si poseen la misma unidad, por ejemplo:

`m1 = Medicion(3.2, "V")` , `m2 = Medicion(1.8, "V")` , `m3 = m1 + m2`

debe generar una nueva instancia equivalente a 5.0 V. Si las unidades son diferentes, el método debe informar que las mediciones no son compatibles y no debe realizar la suma.

6.11. Analice el siguiente código sin ejecutarlo:

```
class Sensor:
    def __init__(self, nombre, valor):
        self.nombre = nombre
        self.valor = valor

    def actualizar(self, incremento):
        self.valor += incremento

    def __str__(self):
        return f"{self.nombre}: {self.valor}"

s1 = Sensor("A", 10)
s2 = Sensor("B", 5)

s1.actualizar(3)
s2.actualizar(s1.valor)

print(s1)
print(s2)
```

- ¿Cuál es el valor final de `s1.valor`?
- ¿Cuál es el valor final de `s2.valor`?
- ¿Qué imprime cada instrucción `print()`?
- ¿Qué método se ejecuta implícitamente al realizar `print(s1)`?
- ¿Los cambios realizados sobre `s1` afectan directamente a `s2`? Explique.

6.12. Analice el siguiente código sin ejecutarlo:

```
class Equipo:
    def estado(self):
        return "Estado general"

class Router(Equipo):
    def estado(self):
        return "Conectado"

class Switch(Equipo):
    pass
```

```
equipos = [Router(), Switch(), Router()]

for equipo in equipos:
    print(equipo.estado())
```

- Escriba exactamente qué imprime el programa.
- ¿Qué versión del método estado() utiliza cada objeto?
- ¿Por qué Switch puede utilizar estado() aunque no lo tenga definido directamente?
- ¿En qué parte del código se evidencia la herencia?
- ¿En qué parte se evidencia el polimorfismo?

6.13. Complete las partes faltantes (las partes _____) para que el programa funcione correctamente (sin ejecutarlo).

```
class Dispositivo:
    def __init__(self, nombre):
        self.nombre = nombre

    def operar(self):
        return "Operación general"

class Sensor(_____):

    def __init__(self, nombre, unidad):
        _____
        self.unidad = unidad

    def operar(self):
        return f"{self.nombre} mide en {self.unidad}"

s = Sensor("Temperatura", "°C")
print(_____)
```

- Complete los tres espacios faltantes.
- Escriba exactamente qué imprime el programa una vez completado correctamente.
- ¿Cuál clase es la clase padre y cuál es la clase hija?
- ¿Qué instrucción permite inicializar el atributo nombre heredado de la clase padre?
- Explique por qué al ejecutar s.operar() se utiliza el método definido en Sensor y no el definido en Dispositivo.

6.14. Depuración: El siguiente código pretende crear un objeto que represente un sensor, pero contiene errores:

```
class Sensor:
    def init(self, nombre, valor):
        nombre = nombre
        valor = valor

    def mostrar(self):
        return f"{self.nombre}: {self.valor}"
```

```
s1 = Sensor("Temperatura", 25)
print(s1.mostrar())
```

- Identifique los errores.
- Explique por qué el programa no funciona correctamente.
- Corrija el código sin modificar el objetivo del programa.

6.15. Depuración:

```
class Persona:
    def __init__(self, nombre):
        self.nombre = nombre

class Estudiante(Persona):
    def __init__(self, nombre, codigo):
        self.codigo = codigo

    def mostrar(self):
        return f"{self.nombre} - {self.codigo}"

e1 = Estudiante("Laura", "A102")
print(e1.mostrar())
```

- Explique por qué se produce un error.
- ¿Qué atributo no fue inicializado?
- Modifique únicamente el constructor de Estudiante para corregir el problema.
- Utilice `super()` en la solución.

6.16. Depuración:

```
class Vector:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, otro):
        x = self.x + otro.x
        y = self.y + otro.y
        return (x, y)

v1 = Vector(2, 3)
v2 = Vector(4, 1)

v3 = v1 + v2
print(v3.x)
```

- La suma se ejecuta, pero posteriormente aparece un error. ¿Por qué?
- ¿Qué tipo de dato está retornando `__add__()`?
- ¿Qué debería retornar para que `v3` siga siendo un objeto de tipo `Vector`?
- Corrija únicamente el método `__add__()`.

	GUÍA DE TRABAJO PRÁCTICO - EXPERIMENTAL Talleres y Laboratorios de Docencia ITM	Código	FGL 029
		Versión	03
		Fecha	18-07-2023

7. PARÁMETROS PARA ELABORACIÓN DEL INFORME

No aplica

8. DISPOSICIÓN DE RESIDUOS

No aplica

9. BIBLIOGRAFÍA

[1] https://github.com/estebangonzalezITM/ProgramacionAvanzada_2026

Elaborado por:	<i>Esteban Gonzalez Valencia</i>
Revisado por:	<i>Comité de Inteligencia Computacional</i>
Versión:	<i>1</i>
Fecha:	<i>Septiembre de 2026</i>